

■ 1.4.10 The Limits of *Mathematica*

In just one *Mathematica* command, you can easily specify a calculation that is far too complicated for any computer to do. For example, you could ask for `Expand[(1+x)^(10^100)]`. The result of this calculation would have $10^{100}+1$ terms – more than the total number of particles in the universe.

You should have no trouble working out `Expand[(1+x)^100]` on any computer that can run *Mathematica*. But as you increase the exponent of $(1+x)$, the results you get will eventually become too big for your computer's memory to hold. Exactly at what point this happens depends not only on the total amount of memory your computer has, but often also on such details as what other processes happen to be running when you try to do your calculation.

If you do run out of memory in the middle of a calculation, most versions of *Mathematica* have no choice but to stop immediately. As a result, it is important to plan your calculations so that they never need more memory than your computer has.

Even if the result of an algebraic calculation is quite simple, the intermediate expressions that you generate in the course of the calculation can be very complicated. This means that even if the final result is small, the intermediate parts of a calculation can be too big for your computer to handle. If this happens, you can usually break your calculation into pieces, and succeed in doing all the pieces on their own. You should know that the internal scheme which *Mathematica* uses for memory management is such that once part of a calculation is finished, the memory used to store intermediate expressions that arose is immediately made available for new expressions.

Memory space is the most common limiting factor in *Mathematica* calculations. Time can also, however, be a limiting factor. You will usually be prepared to wait a second, or even a minute, for the result of a calculation. But you will less often be prepared to wait an hour or a day, and you will almost never be able to wait a year.

One class of calculations where time is often the limiting factor are those that effectively involve searching or testing a large number of possibilities. Integer factorization is a classic example. At some level, the problem of factoring an integer always seems to boil down to something related to testing a large number of candidate factors. As far as we know now, the number of cases to test can increase almost as fast as the exponential of the number of digits in the integer we are trying to factor. As a result, the time needed to factor integers can increase very rapidly with the size of the integers you try to factor. In practice, you will find that `FactorInteger[k]` will give a result almost immediately when k has, say, less

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

than 15 digits. But when k has 40 digits, `FactorInteger[k]` will often take an unmanageably long time.

In the field of computational complexity theory, an important distinction is drawn between algorithms which are “polynomial time”, and those which are not.

A polynomial time algorithm can always be executed in a time that increases only like a polynomial in the length of the input. Non-polynomial time algorithms may take times that increase exponentially with the length of their input.

The internal code of *Mathematica* uses polynomial time algorithms whenever they are known. There are some problems, however, for which no polynomial time algorithms are known. In such cases, *Mathematica* has no choice but to use non-polynomial time algorithms, which may take times that increase exponentially with the length of their input. Integer factorization is one such case. Other cases include factoring polynomials and solving equations when the number of variables involved becomes large.

Even when the time needed to do a computation does not increase exponentially, there will always come a point where the computation is too large or time-consuming to do on your particular computer system. As you work with *Mathematica*, you should develop some feeling for the limits on the kinds of calculations you can do in your particular application area.

Doing arithmetic with numbers containing a few thousand digits.

Expanding out a polynomial that gives a few hundred terms.

Factoring a polynomial in two variables with a hundred terms.

Applying a recursive rule a few thousand times.

Finding the numerical inverse of a 50×50 matrix.

Formatting a few pages of output.

Some operations that typically take a few seconds on a 1987 workstation.